Introduction

Reproducible computational research demands that every claim be traceable back to a stable artifact — code, data, and citations alike (Peng 2011). Manual literature curation is a well-known bottleneck in such workflows: a graduate student writing a related-work section may spend hours searching arXiv, Crossref, and Google Scholar; copying citations into a `.bib` file by hand; and tracking which papers they have actually read. Three failure modes recur:

Introduction I

1. **Style drift** — hand-edited `.bib` files accumulate formatting inconsistencies that hide real semantic conflicts in version-control diffs.
2. **Stale state** — the bibliography, the reading list, and the manuscript prose drift apart as the project evolves; the citation key in the manuscript no longer matches the entry in `.bib`, or the entry no longer matches the actual paper.
3. **Lost context** — abstracts and full text are read once during search, then discarded; six months later the same paper has to be re-skimmed to recall its contribution.

`template_search_project` exists to demonstrate one disciplined solution. The pipeline outputs are summarised in sec. `sec:methodology` (overview figure at the start of that section):

Introduction I

- The discovery side (`infrastructure/search/`) provides multi-source paper search with failure-isolated aggregation, DOI/arXiv-aware deduplication, and deterministic JSON caching keyed on canonical query identity.
- The export side (`infrastructure/reference/`) provides BibTeX read/write/convert facilities byte-compatible with the existing exemplar `references.bib`, suitable for the combined-PDF pipeline (Pandoc `--natbib` + BibTeX).
- A small project-local synthesis layer (in `src/pipeline/synthesis.py`) takes enriched papers, builds reproducible LLM prompts, and assembles a markdown reading report.

The project is *configurable* via a single `manuscript/config.yaml`: changing the topic, year filters, backend set, enrichment level, and LLM parameters never requires editing code. The project is *modular* in the strict sense the template uses: every reusable component lives in `infrastructure/`, and `src/` contains only project-specific orchestration.

The contribution of this exemplar is therefore not a new algorithm; it is a **demonstration that a reproducible literature workflow can be built from existing template infrastructure** with no new optional dependencies, no mocks in the test suite, and complete configurability through a single YAML file.